

# FinShare – Enhanced Auth & Security

Backend Requirements | For: Abhilash Thomas | April 3, 2026

## Overview

This document covers all backend work needed for the next phase of FinShare authentication and security. There are 5 areas:

- Email verification after registration (confirm the email is real)
- Mobile OTP option (optional – Phase 2)
- Strong password policy enforcement on the server
- Forgot password + reset password link flow (referenced from previous note, included here for completeness)
- Biometric / WebAuthn login – backend credential storage (when ready)

## Current Auth Status

| Feature         | Status          | Notes                                     |
|-----------------|-----------------|-------------------------------------------|
| Register        | ✅ Working       | Endpoint exists, field names fixed        |
| Login           | ✅ Working       | JWT returned, stored in localStorage      |
| Logout          | ✅ Frontend only | Clears finshare_token – no backend needed |
| Email Verify    | ❌ Missing       | New – see Section 1 below                 |
| Forgot Password | ❌ Missing       | New – see Section 3 below                 |
| Reset Password  | ❌ Missing       | New – see Section 3 below                 |
| Strong Password | ❌ Missing       | New – see Section 2 below                 |
| Biometric Login | 🕒 Phase 2       | Plan now, build when frontend is ready    |

## Section 1 – Email Verification on Registration

After a user registers, they should receive an email with a verification link before they can log in. This prevents fake/mistyped email addresses in the system.

### 1a. Database Changes

```
ALTER TABLE Users
  ADD IsEmailVerified BIT NOT NULL DEFAULT 0,
  EmailVerifyToken NVARCHAR(200) NULL,
  EmailVerifyExpiry DATETIME NULL;
```

### 1b. Stored Procedure – sp\_SaveEmailVerifyToken

```

CREATE OR ALTER PROCEDURE sp_SaveEmailVerifyToken
    @Email      NVARCHAR(200),
    @Token      NVARCHAR(200),
    @ExpiryHours INT = 24
AS BEGIN
    UPDATE Users
        SET EmailVerifyToken = @Token,
            EmailVerifyExpiry = DATEADD(HOUR, @ExpiryHours, GETUTCDATE())
        WHERE EmailAddress = @Email;
END

```

### 1c. Stored Procedure – sp\_VerifyEmail

```

CREATE OR ALTER PROCEDURE sp_VerifyEmail
    @Token NVARCHAR(200)
AS BEGIN
    DECLARE @UserId INT, @Expiry DATETIME;

    SELECT @UserId = MemberId, @Expiry = EmailVerifyExpiry
        FROM Users WHERE EmailVerifyToken = @Token;

    IF @UserId IS NULL
        SELECT 0 AS Success, 'Invalid token' AS Message;
    ELSE IF @Expiry < GETUTCDATE()
        SELECT 0 AS Success, 'Link expired' AS Message;
    ELSE BEGIN
        UPDATE Users
            SET IsEmailVerified = 1,
                EmailVerifyToken = NULL,
                EmailVerifyExpiry = NULL
            WHERE MemberId = @UserId;
        SELECT 1 AS Success, 'Email verified' AS Message;
    END
END

```

### 1d. Endpoint – POST /api/Auth/verify-email

#### Endpoint Spec: POST /api/Auth/verify-email

```

Request  : { token: string }
Response : { success: true, message: 'Email verified successfully' }
           { success: false, message: 'Invalid or expired link' }

Logic:
1. Call sp_VerifyEmail(@Token)
2. If success = 1 → return 200 OK
3. If success = 0 → return 400 Bad Request with message
   [No auth required – public endpoint]

```

### 1e. Update Register Endpoint

The existing /api/Auth/register endpoint needs two small changes:

- After creating the user, generate a token (Guid.NewGuid().ToString("N")), store it via sp\_SaveEmailVerifyToken, then send the verification email.
- Do NOT return a JWT on registration anymore. Return: { success: true, message: 'Registration successful. Please check your email to verify your account.' }

### 1f. Update Login Endpoint

Login must now check IsEmailVerified. If 0, block login with a friendly message:

```
// In sp_ValidateUser or in the controller after validation:
IF IsEmailVerified = 0
    RETURN 'Please verify your email before logging in.'
```

### 1g. Verification Email Template

| Email: Verify your FinShare account                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Subject : Verify your FinShare email address</p> <p>Body:</p> <p>Hi {MemberName},</p> <p>Welcome to FinShare! Please click the link below to verify your email address:</p> <p><a href="https://finshare.me/verify-email?token={TOKEN}">https://finshare.me/verify-email?token={TOKEN}</a></p> <p>This link expires in 24 hours.</p> <p>If you did not create a FinShare account, please ignore this email.</p> <p>- The FinShare Team</p> |

## Section 2 – Strong Password Policy

Apply password validation on the server side in both Register and Reset Password endpoints. The frontend will also show a strength indicator (Manus will handle that), but the backend must also enforce the rules so they cannot be bypassed.

### 2a. Password Rules

| Rule              | Requirement                        |
|-------------------|------------------------------------|
| Minimum length    | 8 characters                       |
| Uppercase letter  | At least 1 (A-Z)                   |
| Lowercase letter  | At least 1 (a-z)                   |
| Number            | At least 1 (0-9)                   |
| Special character | At least 1 (!@#\$\$%^&*-_+=)       |
| Maximum length    | 64 characters                      |
| No spaces         | Reject passwords containing spaces |

### 2b. C# Validator Utility

```
// Add this class: Services/PasswordValidator.cs
public static class PasswordValidator
{
    public static (bool Valid, string Message) Validate(string password)
    {
```

```

        if (string.IsNullOrEmpty(password) || password.Length < 8)
            return (false, "Password must be at least 8 characters.");
        if (password.Length > 64)
            return (false, "Password must not exceed 64 characters.");
        if (password.Contains(' '))
            return (false, "Password must not contain spaces.");
        if (!password.Any(char.IsUpper))
            return (false, "Password must contain at least one uppercase
letter.");
        if (!password.Any(char.IsLower))
            return (false, "Password must contain at least one lowercase
letter.");
        if (!password.Any(char.IsDigit))
            return (false, "Password must contain at least one number.");
        var special = "!@#$$%^&*-_+=";
        if (!password.Any(c => special.Contains(c)))
            return (false, "Password must contain at least one special
character.");
        return (true, "Password is strong.");
    }
}

```

Use in Register and Reset Password endpoints:

```

var (valid, msg) = PasswordValidator.Validate(request.Password);
if (!valid) return BadRequest(new { success = false, message = msg });

```

## Section 3 – Forgot Password & Reset Password

These were detailed in the previous note (FinShare\_ForAbhilash\_ForgotPassword\_v2.docx).

Here is a quick summary for completeness:

| Item                           | Detail                                                                                                                                       |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| POST /api/Auth/forgot-password | Takes {email}. Generates secure token, stores in DB, sends reset link via email. Always returns success (don't expose whether email exists). |
| POST /api/Auth/reset-password  | Takes {token, newPassword}. Validates token + expiry, applies strong password check, updates BCrypt hash, clears token fields.               |
| DB columns                     | PasswordResetToken NVARCHAR(200) NULL,<br>PasswordResetExpiry DATETIME NULL                                                                  |
| Reset link URL                 | <a href="https://finshare.me/reset-password?token=XXXX">https://finshare.me/reset-password?token=XXXX</a>                                    |
| Token expiry                   | 1 hour                                                                                                                                       |

Please refer to the previous document for full SP code and email template.

## Section 4 – Biometric Login (WebAuthn / FIDO2)

### 4a. Good News – WebAuthn Works in PWA!

You mentioned you think biometric only works in Flutter. Actually, the Web Authentication API (WebAuthn) is a W3C standard supported by all modern browsers, including in PWA mode. Here is what works:

| Platform               | Status                                             |
|------------------------|----------------------------------------------------|
| iOS Safari             | ✅ Face ID and Touch ID — works perfectly           |
| Android Chrome         | ✅ Fingerprint and face unlock — works perfectly    |
| macOS Safari/Chrome    | ✅ Touch ID — works                                 |
| Windows Edge/Chrome    | ✅ Windows Hello (face, fingerprint, PIN)           |
| iOS Chrome             | ⚠️ Limited (uses WebKit) — recommend Safari on iOS |
| Older devices pre-2018 | ❌ No support                                       |

Flutter's advantage is a more consistent native-feel UX and works offline. For FinShare as a PWA, WebAuthn is production-ready and will cover most modern users well.

#### 4b. How It Works (Two Phases)

Phase 1 – Registration (done once, after normal login):

- Frontend calls: POST /api/Auth/webauthn/register-challenge
- Backend generates a random challenge and returns it
- Browser calls navigator.credentials.create() → device shows Face ID / fingerprint prompt
- User approves → browser returns a public key credential
- Frontend sends credential to: POST /api/Auth/webauthn/register
- Backend stores the credential ID and public key against the user's account

Phase 2 – Authentication (on subsequent logins):

- Frontend calls: POST /api/Auth/webauthn/auth-challenge
- Backend generates a challenge and returns it
- Browser calls navigator.credentials.get() → device shows Face ID / fingerprint prompt
- User approves → browser returns a signed assertion
- Frontend sends assertion to: POST /api/Auth/webauthn/auth
- Backend verifies the signature using stored public key → returns JWT

#### 4c. NuGet Package

| Recommended: Fido2NetLib                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Install-Package Fido2NetLib<br>This handles all the cryptographic verification for you.<br>GitHub: <a href="https://github.com/passwordless/fido2-net-lib">passwordless/fido2-net-lib</a><br>Docs: <a href="https://fido2-net-lib.passwordless.dev/">https://fido2-net-lib.passwordless.dev/</a> |

#### 4d. Database Changes for WebAuthn

```
-- Add to Users table:  
ALTER TABLE Users  
ADD WebAuthnCredentialId NVARCHAR(500) NULL,
```

```

WebAuthnPublicKey    NVARCHAR(MAX) NULL,
WebAuthnSignCount    BIGINT          NULL DEFAULT 0;

-- Or create a separate table (better if one user can have multiple devices):
CREATE TABLE UserWebAuthnCredentials (
    Id                INT                IDENTITY PRIMARY KEY,
    MemberId          INT                NOT NULL REFERENCES Users (MemberId),
    CredentialId       NVARCHAR(500)     NOT NULL,
    PublicKey          NVARCHAR(MAX)     NOT NULL,
    SignCount          BIGINT             NOT NULL DEFAULT 0,
    DeviceName         NVARCHAR(100)     NULL,    -- e.g. 'iPhone 15 Face ID'
    CreatedAt          DATETIME          NOT NULL DEFAULT GETUTCDATE()
);

```

#### 4e. Endpoints to Create

| Endpoint                                   | Description                                                                                           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| POST /api/Auth/webauthn/register-challenge | Returns: { challenge, userId, userName }. Store challenge in session/cache with short expiry (2 min). |
| POST /api/Auth/webauthn/register           | Body: { credential }. Verify with Fido2NetLib, store credentialId + publicKey in DB.                  |
| POST /api/Auth/webauthn/auth-challenge     | Returns: { challenge, allowCredentials }. Store challenge in cache.                                   |
| POST /api/Auth/webauthn/auth               | Body: { assertion }. Verify signature, update signCount, return JWT.                                  |

Note: All WebAuthn endpoints require HTTPS. The app is already on <https://fintrack.sysdatacenter.com> so this is fine.

This is Phase 2 work – do not rush it. Let Manus finish the frontend WebAuthn code first, then we can align on the exact request/response shapes.

## Section 5 – Mobile OTP (Optional / Phase 3)

Mobile OTP (one-time password to phone number) is an optional alternative to email verification. Recommendation: start with email verification (Section 1) first as it has no cost. Add SMS OTP as a Phase 3 enhancement.

#### 5a. SMS Provider Options

| Provider  | Notes                                                                                   |
|-----------|-----------------------------------------------------------------------------------------|
| Twilio    | ✅ Most popular. Easy .NET SDK. Pay per SMS (~\$0.0079/message). Free trial available.   |
| MSG91     | ✅ Popular in India. Competitive pricing for Indian numbers. Good for FinShare's market. |
| Azure SMS | ✅ If already using Azure. Integrated with Azure Communication Services.                 |
| Fast2SMS  | ✅ India-only. Very cheap (INR 0.14/SMS). Good for Indian users.                         |

## 5b. If You Proceed with OTP

If you want to add OTP, the DB and SP structure would be:

```
ALTER TABLE Users ADD MobileVerified BIT NOT NULL DEFAULT 0;

CREATE TABLE OTPVerifications (
    Id          INT IDENTITY PRIMARY KEY,
    Mobile      NVARCHAR(20) NOT NULL,
    OTPCode     NVARCHAR(10) NOT NULL,
    Expiry      DATETIME     NOT NULL,
    IsUsed      BIT          NOT NULL DEFAULT 0,
    CreatedAt   DATETIME     NOT NULL DEFAULT GETUTCDATE()
);
```

Endpoints needed: POST /api/Auth/send-otp (sends 6-digit OTP to mobile) and POST /api/Auth/verify-otp (validates code, marks mobile as verified).

## Summary – What to Build

| #  | Task                                                    | Priority           |
|----|---------------------------------------------------------|--------------------|
| 1  | ALTER TABLE – add email verify columns                  | Phase 1 – first    |
| 2  | sp_SaveEmailVerifyToken + sp_VerifyEmail                | Phase 1            |
| 3  | POST /api/Auth/verify-email endpoint                    | Phase 1            |
| 4  | Update /register – no JWT on return, send verify email  | Phase 1            |
| 5  | Update /login – block if IsEmailVerified = 0            | Phase 1            |
| 6  | PasswordValidator.cs utility class                      | Phase 1            |
| 7  | ALTER TABLE – reset password columns (see previous doc) | Phase 1            |
| 8  | sp_SavePasswordResetToken + sp_ResetPassword            | Phase 1            |
| 9  | POST /api/Auth/forgot-password + reset-password         | Phase 1            |
| 10 | SendGrid or SMTP email service                          | Phase 1            |
| 11 | ALTER TABLE / CREATE TABLE for WebAuthn credentials     | Phase 2            |
| 12 | Install Fido2NetLib, create 4 WebAuthn endpoints        | Phase 2            |
| 13 | SMS OTP (Twilio/MSG91)                                  | Phase 3 – optional |